Try to find better solution compared with RSAD

1. Attempt 1: optimize the code of the min-cost-max-flow based placer.
2. Attempt 2: simplify and transform the problem formulation into a shortest path problem and rewrite the DP based placer.
3. Attempt 3: use external memory instead of RAM (with DP based placer)
4. Attempt 4: try to simplify the DAG and compress the # nodes in DAG.

Attempt 1: optimize the code of the MCMF based placer.

1. Frontier Compression, only keep the active-layer: reduce around 5x memory usage
2. Compact State Hashing: reduce memory usage and speedup graph construction
3. Parallel computing: around 3x acceleration
4. Now it can solve 16*16 problem in 519s with the same HPWL as the RSAD result.

Attempt 2: simplify the problem statement

1. For each edge, cap = 1, and the input flow = 1
2. Then the MCMF problem can be simplified as shortest path problem with.
3. Then DFS/DP should be the one of the best algorithm for this problem.

Attempt 3: use external memory instead of RAM

1. Level: the state (a1,a2,a3,…,am), if sum of the a_i = L, then we say this state is in Level L
2. Only need to store the current level L and level (L-1)
3. Even in one level, we can process them part by part.
4. This method can make the DP based placer solve upto 18*18 problem
5. Now runtime is the bottleneck, 18*18 problem need more than 24 h to solve.
6. Still has the same quality as RSAD

Why we can not win RSAD

1. The DFS is O(V+E) algorithm for shortest path problem in DAG.
2. But DAG has $\#\text{states}(m, h) = \binom{m+h}{m}$
3. If RSAD solution is optimal, then this DAG definitely can be simplified.
4. Cost matrix:
5. Result in lots of edge in DAG with 0 cost.
6. Then this special structure of DAG gives a chance to do state compression

Next step

1. Use the current DP/MCMF based placer handle the max single wirelength<T constraint.
2. T is the parameter.
3. By giving penalty weight to the net with long wirelength.
4. Dreamplace 4.0 handles the WNS by giving weight to net during placement.
5. Extend to multiple DSP site columns

Brief intro

1. Problem formulation
2. Transfer into a shortest path problem
3. Timing-driven extension

Problem formulation

1. Map m x h DSP marco array to one DSP site column.
2. Under Order consistency (OC)
3. (i,j) denotes coordinate of a macro in array. Y(i,j) denotes the coordinate of placed sites for macro(i,j)
4. OC: if $i_a >= i_b$, $j_a >= j_b$; $Y(i_a, j_a) > Y(i_b, j_b)$
5. Objective : HPWL

Transfer into a shortest path problem

1. Due to OC , the order of each pair of macro in any net are determined.
2. Then objective function can be written as
   a. HPWL = sum of |Y(i_a,j_a)- Y(i_b,j_b)| (for all nets)
   b. = sum of (Y(i_a,j_a)- Y(i_b,j_b)) (no abs())
   c. = sum of C_i,j * Y(i,j), (C_i,j) is the sum of all coefficients for Y(i,j)
3. Easy to extend to weighted HPWL
   a. HPWL = sum of w*|Y(i_a,j_a)- Y(i_b,j_b)|
   b. = sum of C_i,j * Y(i,j),

State-DAG

1. State: $a=(a_0,\ldots,a_{m-1})$, with $a_0 \geq \cdots \geq a_{m-1} \in [0,h]$, due to OC
2. Level of state: L= sum of a_i in one state
3. Edge, connecting two state in level L and level (L+1)
4. Weight of edge = (L+1) * the cost of new placed macro
5. State S = (0,0,....,0) : no macro is placed
6. State T = (h,h, …., h) all marco is placed
7. The shortest path from S to T is the solution of the optimal solution of the original problem
8. Can be solved by DP, or min-cost-max-flow solver

Runtime comparison

m	h	DP_HPWL	DP_Time(s)	DP_Peak(MB)	MCMF_HPWL	MCMF_Time(s)	MCMF_Peak(MB)
4	4	60		1.461	60	0.017	3.960
5	5	116		1.539	116	0.021	4.100
6	6	200	0.000	1.480	200	0.020	4.160
7	7	318	0.001	1.477	318	0.023	4.170
8	8	472	0.006	2.906	472	0.028	4.290
9	9	668	0.021	3.109	668	0.047	4.480
10	10	914	0.057	3.844	914	0.096	5.390
11	11	1214	0.260	6.027	1214	0.301	8.440
12	12	1568	1.113	14.246	1568	1.068	22.570
13	13	1988	5.194	62.652	1988	4.837	75.320
14	14	2480	25.049	124.707	2480	18.347	210.180
15	15	3040	113.051	431.379	3040	79.069	868.390
16	16	3680	835.640	812.620	3680	519.440	945.260
avg			89.13	111.91		47.95	162.73
norm(M/D)			100%	100%		53.80%	145.41%

Timing-driven extension

1. Minimize the max delay
2. Weight each net based on some delay relevant internal metrics
3. Internal metrics:
    a. Max net wirelength
        i. Straight-forward, max net length <= T
    b. Delay lookup table
        i. For each (dx,dy), measure the delay, from T-Vplace (ISFPGA 2000)
        ii. Not aware of congestion
    c. STA
        i. slow
    d. Congestion-aware delay lookup table
        i. For each (dx,dy,congestion level), measure the delay
        ii. Need to define congestion level and how to measure

First try on minimize max net wire length

1. Momentum weight policy in log domain
    a. v(k) = max(0, -slack(k))/(clk period), in our case v(k) = max(0, net length - T)
    b. w'(k) = log(w(k))
    c. w'(k+1)-w'(k) =alpha * (w'(k) - w'(k-1) + (1-alpha) * gemma * log(1+ beta * v(k))
    d. Normalize the weight: w(k)_norm = w(k)/ median of all w(k)
2. Problem:
    a. On 16*8 systolic array testcase, strong numerical oscillation, hard to converge

Why this policy works on dreamplace

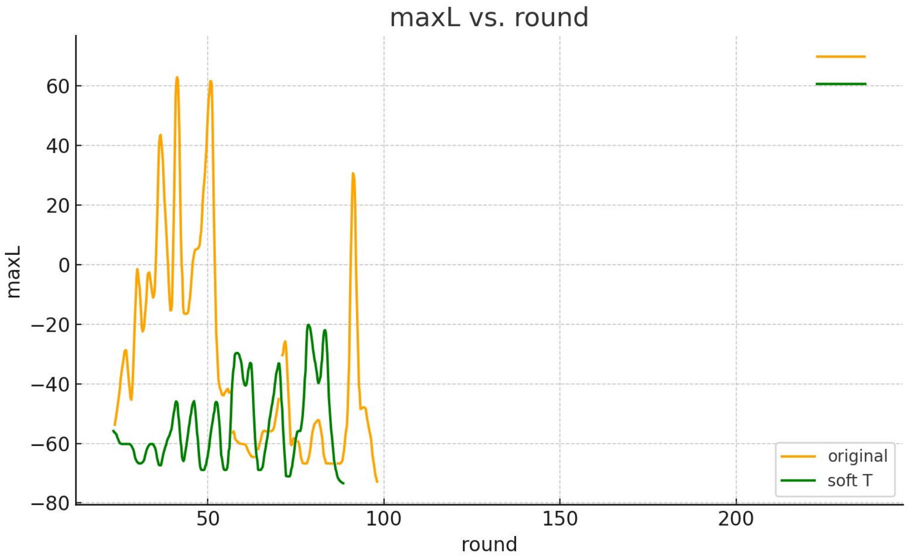
1. There is a spreading out process in dreamplace
2. Initial WNS is small, so the timing weights are also small.
3. But in our case, there is no spreading out process.
4. We need to find a way to have small changes of weight during iterations.

Possible solutions

1. Relax the max T to T’ constraints at beginning, and tighten it when the loose T’ is satisfied. I call it soft T strategy
2. T’ = inf before round 1
3. T’ is initialized as max WL after round 1
4. T’ *= max(T, 0.8*T’) once T’ is satisfied.

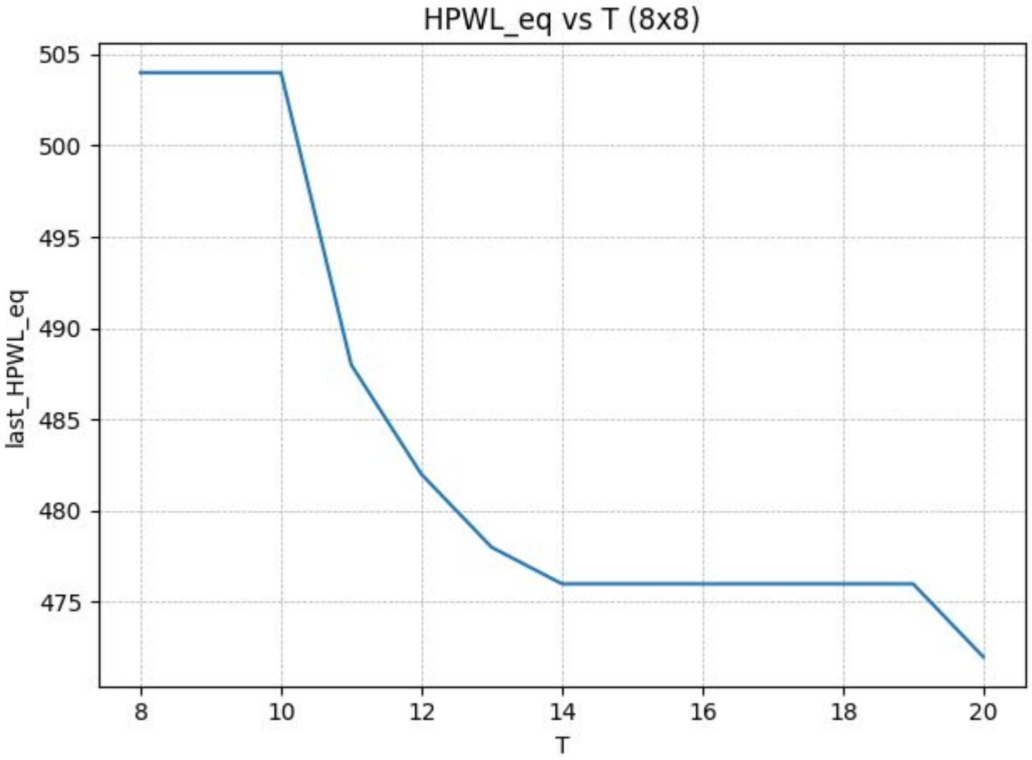
comparison

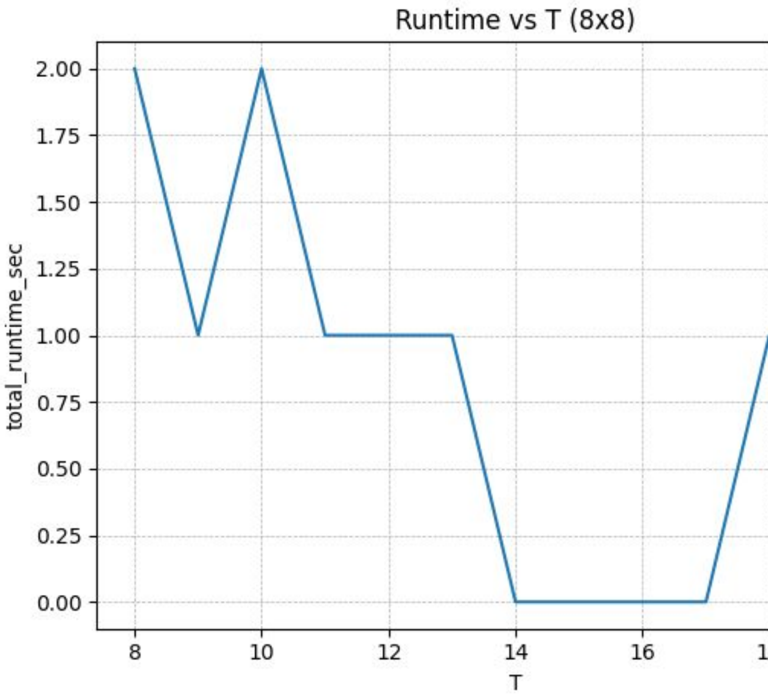
1. RSAD, HPWL=1040, MAX_L=18; our HPWL=1050, MAX_L=12
2. 0.9% HPWL increase, 33% MAX_L decrease



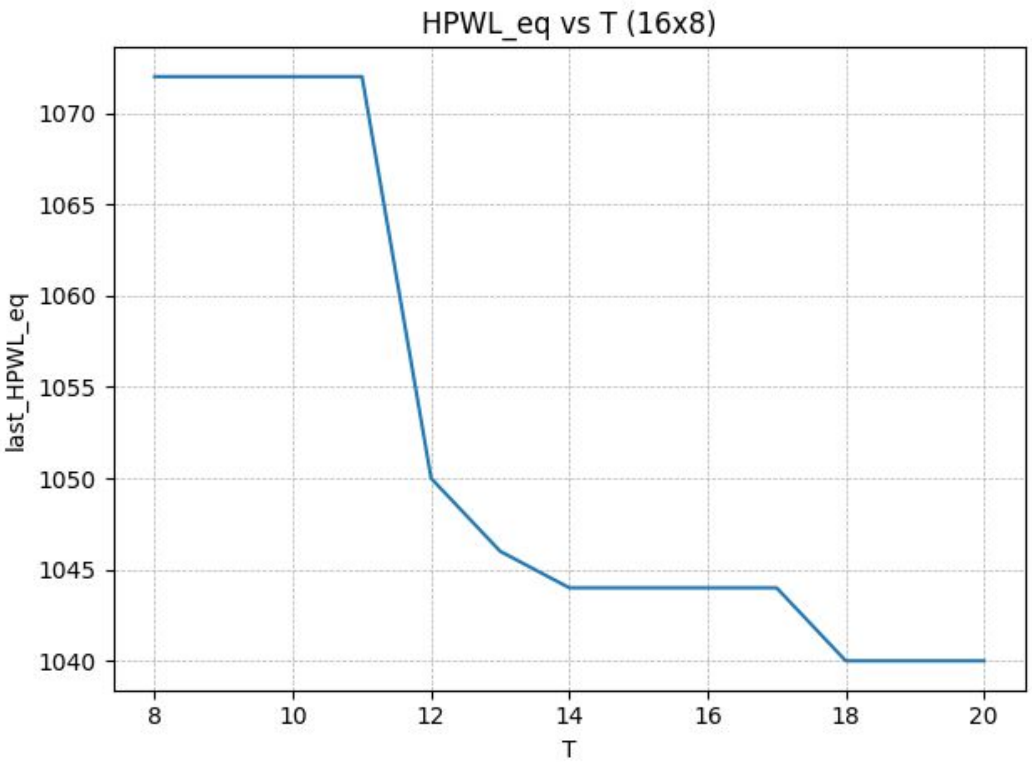
policy	maxL_max	round_of_maxL_max	maxL_min	round_of_maxL_min	HPWL_eq_final	HPWL_w_final	runtime_se
original	82	12	12	47	1088	7607000	2
soft	38	35	12	41	1050	1223741	1

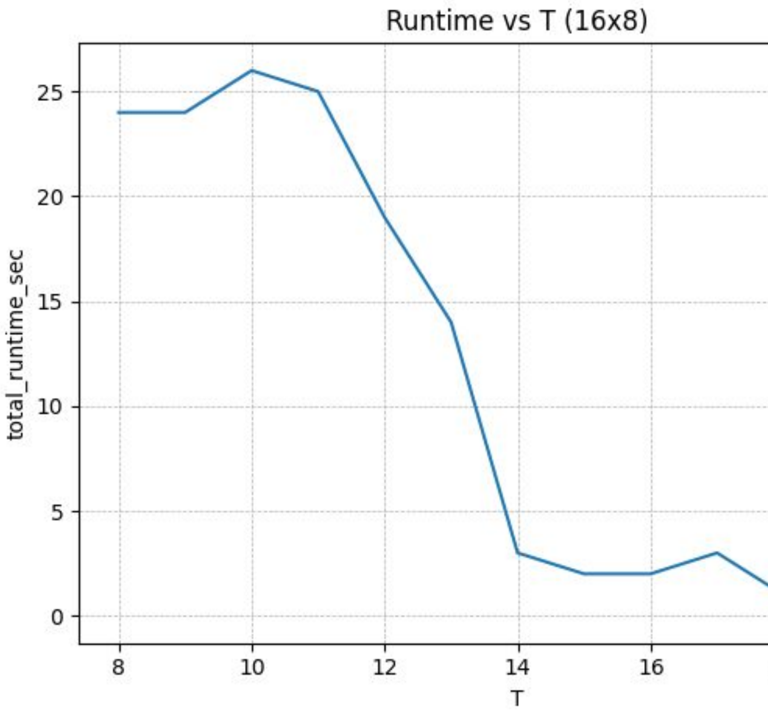
For all T, our method can have a feasible solution



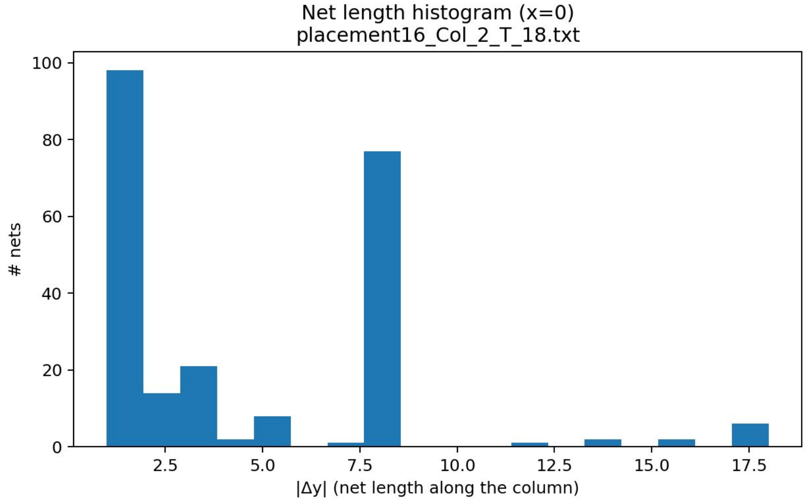


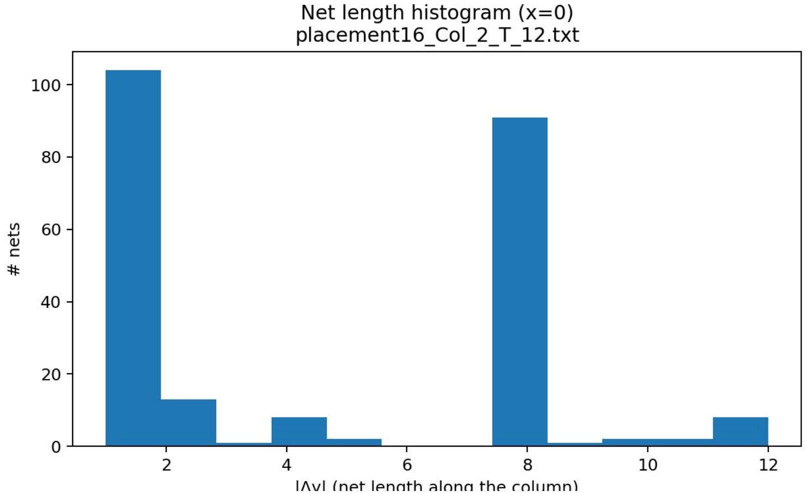
For 16*8, 32*20 32*5

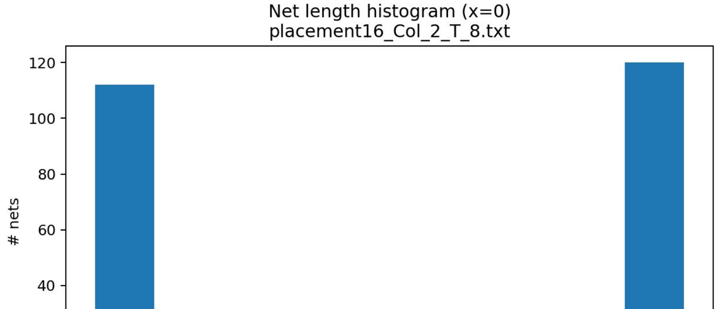




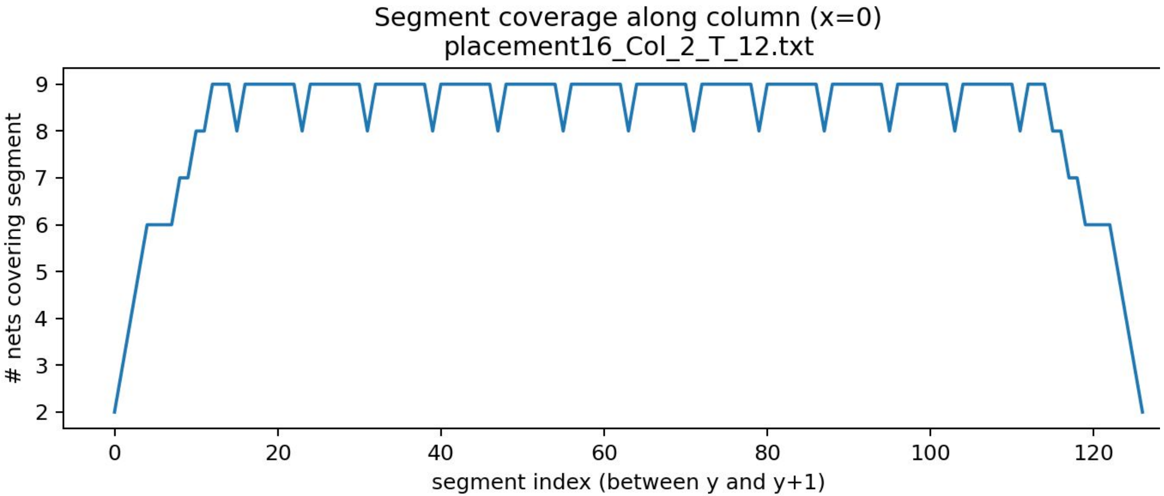
Net wirelength distribution

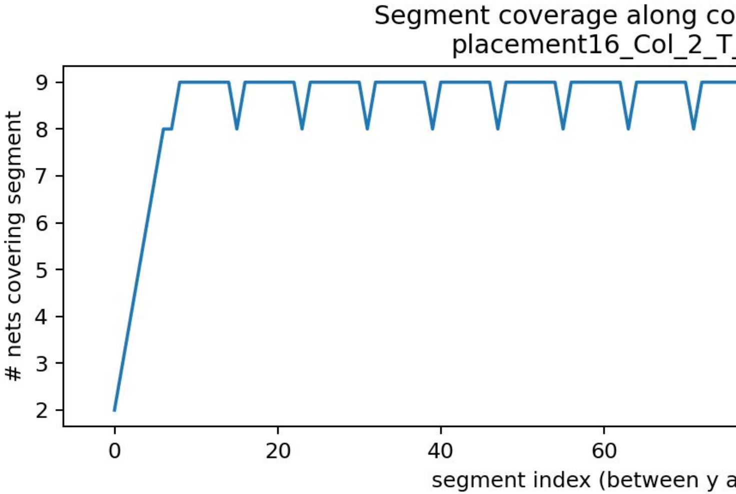


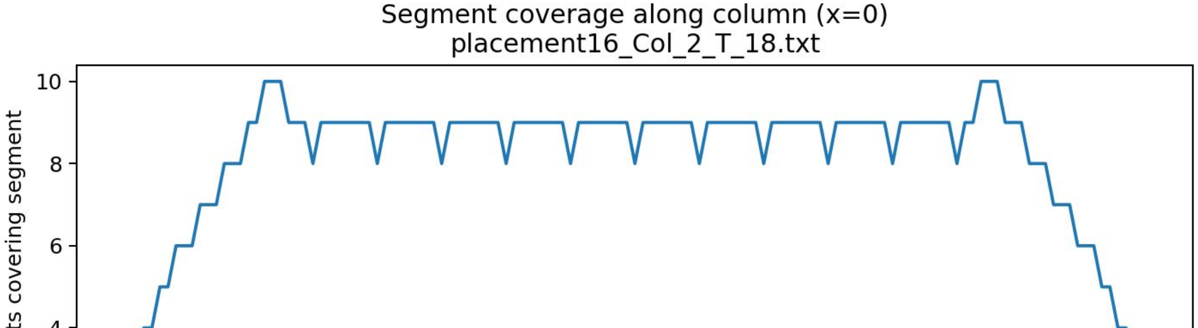




Congestion distribution







Proof of OC not harmful

1. Target: proof that when all net has identical weight,

   the optimal solution under OC constraints

   Is the local optimal solution without OC constraint.

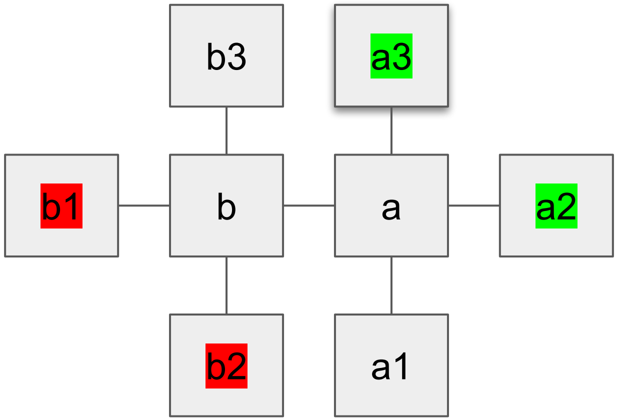
2. Distance k 2-swap definition
   a. For macro A (i_a,j_a) and macro B (i_b,j_b), if |i_a - i_b| +|j_a-j_b| = k
   b. Then the 2-swap of A and B is so-called distance k 2-swap
3. Proof by induction
   a. Step 1: any “distance 1” 2-swap of opt solution under OC will not decrease the HPWL
   b. Step 2: if “distance k” 2-swap will not help, then “distance k+1” 2-swap will not help neither.
   c. Step 3: any 2-swap will not decrease the HPWL

Step 1: any “distance 1” 2-swap of opt solution under OC

1. If this 2-swap is not breaking OC constraint,
2. HPWL will not decrease since this is opt solution under OC
3. If this distance 1 2-swap is against OC, there are 4 situations.

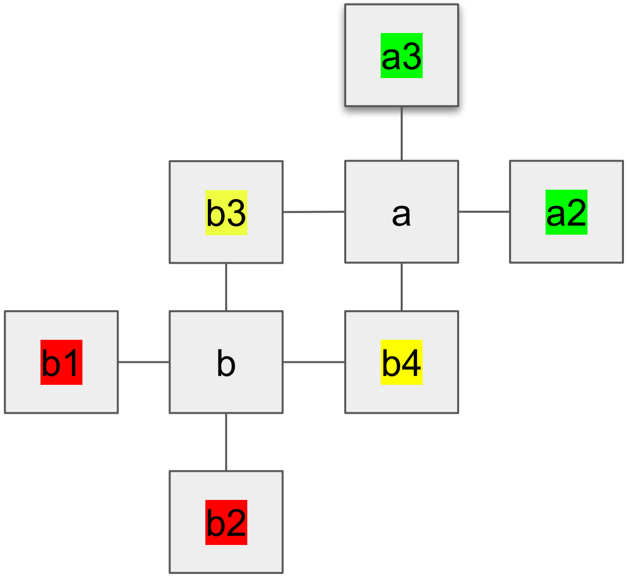
Situation 1: macro A and B both are not on the boundary

1. If i_a = i_b+1
2. Before swap, we have
   a. Y(b1) or Y(b2) <
      Y(b)<Y(a)<Y(a3) or Y(a2)
   b. D = Y(a) - Y(b)
3. After swap
   a. Net b1-b, b2-b, a-a3, a-a2 increase D for each
   b. Net b-a keep the same
   c. Net b3-b, a-a1 not sure
4. HPWL_after-HPWL_before
   >= +4D+0D-2D=2D



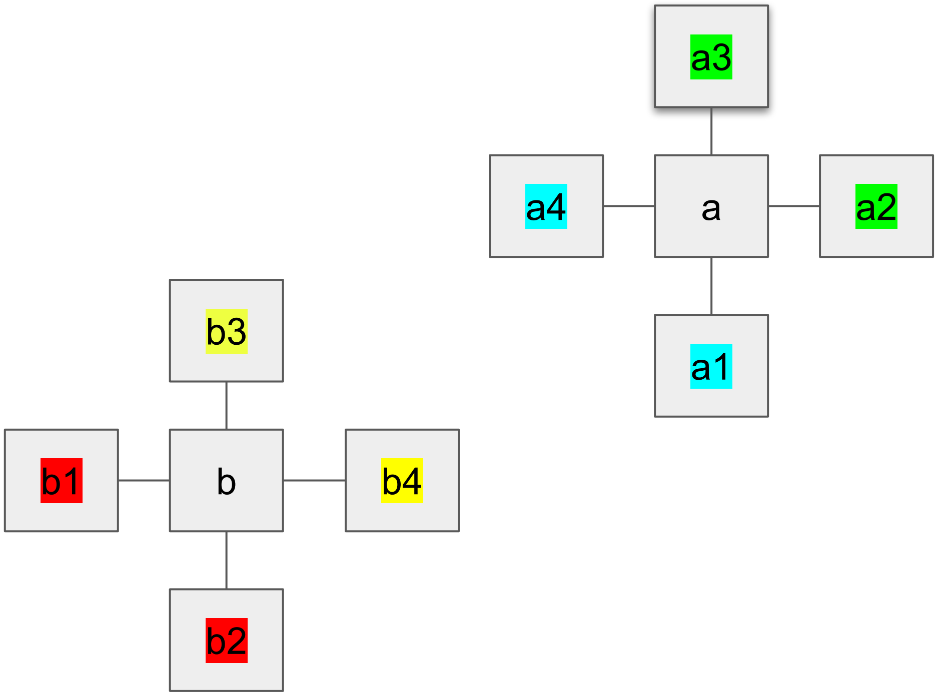
Situation 1: macro A and B both are not on the boundary

1. If distance = 2
    a. +4D



Situation 1: macro A and B both are not on the boundary

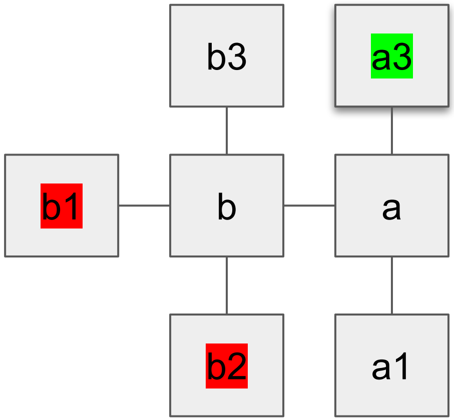
1. If i_a>ib, j_a>j_b
   a. Y(b1) or Y(b2) < Y(b)< Y(b3) or Y(b4) < Y(a4) or Y(a1)<Y(a)<Y(a3) or Y(a2)
   b. B-b1,b-b2,a-a3,a-a2 +4D
   c. Before Y(a)-Y(a4) + Y(a)-Y(a1)+Y(b3)-Y(b)+Y(b4)-Y(b)
   d. =2Y(a)-2Y(b)+Y(b3)+Y(b4)-Y(a4)-Y(a1)
   e. After 2Y(a) - 2Y(b) -Y(b3)-Y(b4) +Y(a4)-Y(a1)
   f. After-before = 2(Y(a4)+Y(a1)-Y(b3)-Y(b4)) >=8



Situation 2: A is on boundary and B is not

3 nets increase after swap, 2 may decrease, and 1 keeps the same.

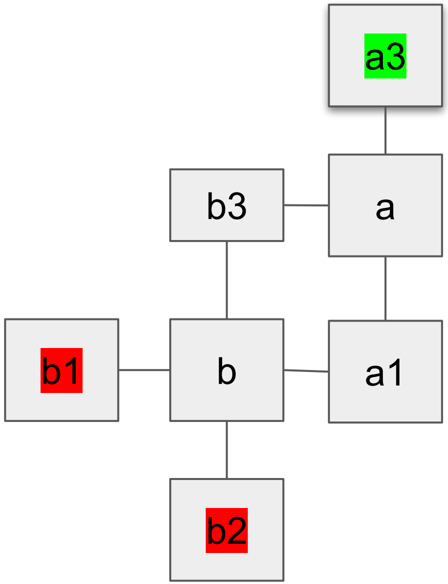
In total at least increase 1D



Situation 2: A is on boundary and B is not

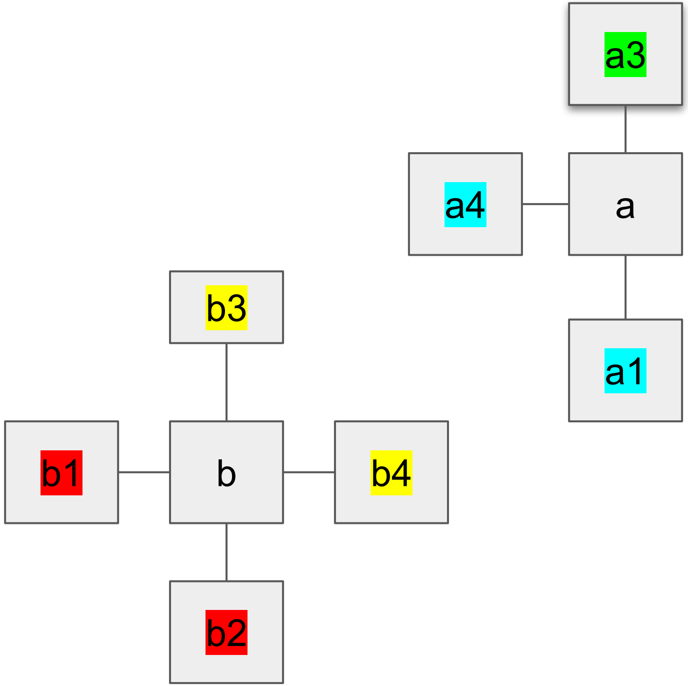
Distance = 2

+3D



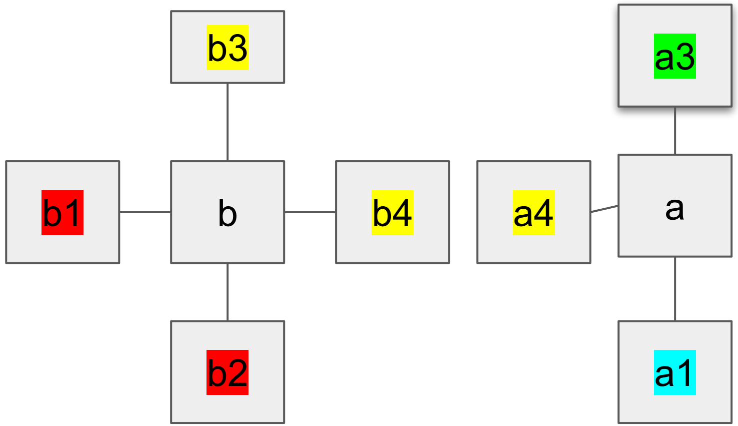
Situation 2: A is on boundary and B is not

1. If i_a>ib, j_a>j_b,
2. +3D+ >=8, according to previous part



Situation 2: A is on boundary and B is not

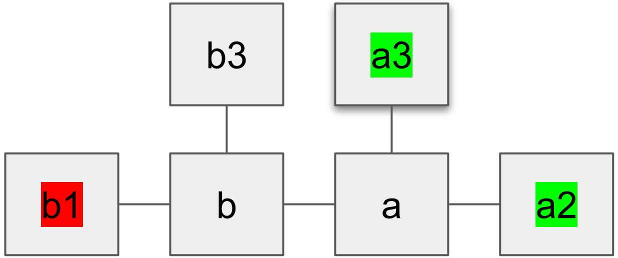
1. If same row
2. +1D
3. B-b4, a4-a
4. before=Y(b4)-Y(b)
   +Y(a)-Y(a4)
5. After =
   Y(a)-Y(b4)+Y(b4)-Y(b)
6. After - before =
   2(Y(a4)-Y(b4)) >=0



Situation 3: A B are on boundary

3 increase, 1 may decrease, 1 keeps the same.

At least 2D increase.

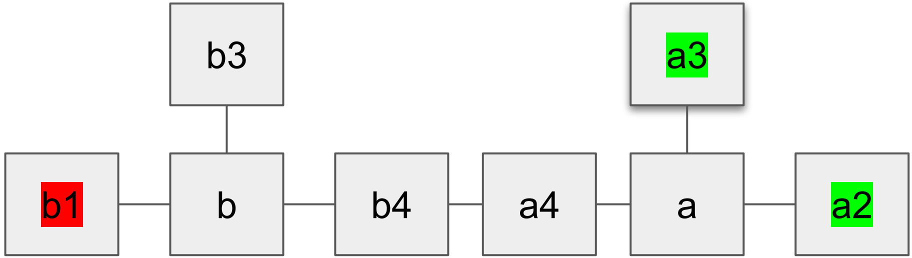


Situation 3: A B are on boundary

+3D,

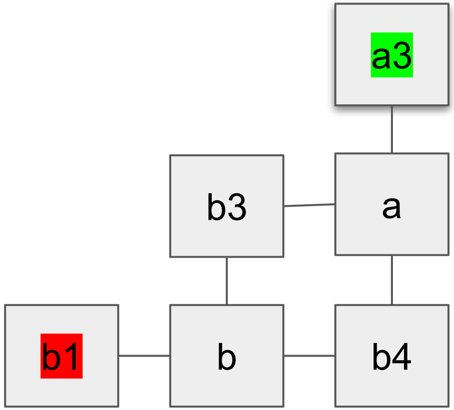
May be -1D

after-before(b-b4) (a-a4) >=0



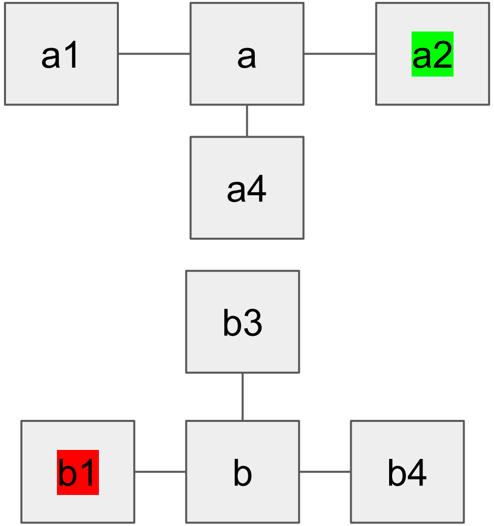
Situation 3: A B are on boundary

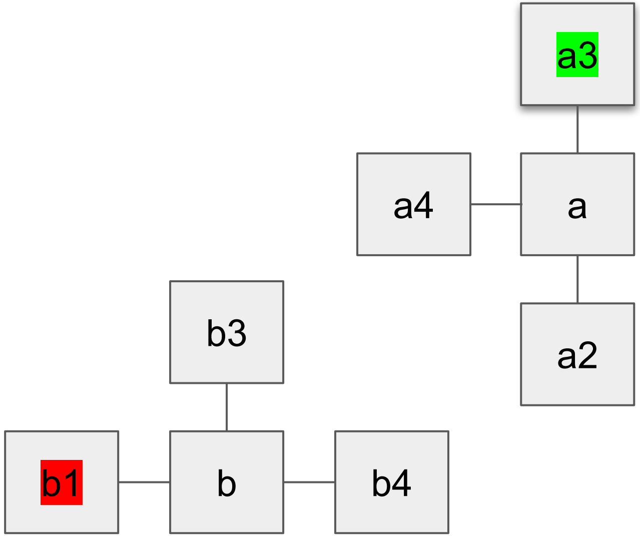
+2D



Situation 3: A B are on boundary

+2D+>=8

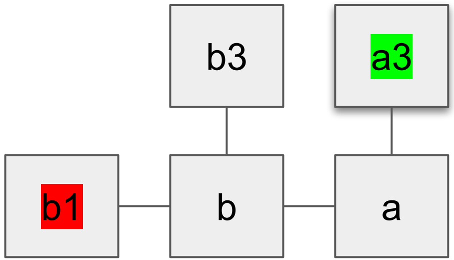




Situation 4: A is at the corner, and B is on the boundary

2 increase, 1 may decrease, 1 keeps the same

At least 1D increase.

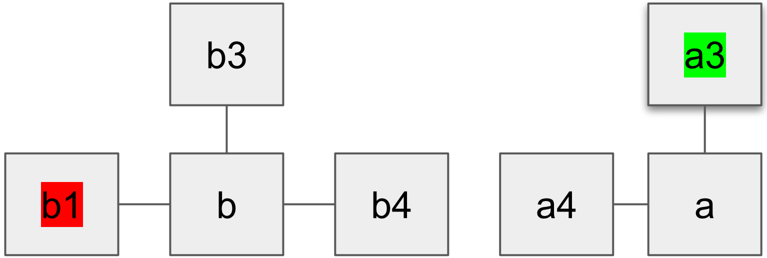


Situation 4: A is at the corner, and B is on the boundary

Same row,+2D

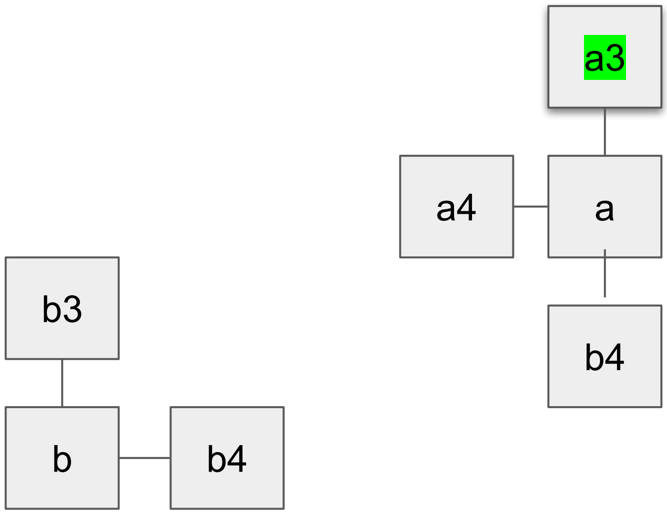
May be -D

B-b4 a-a4>=0



Situation 4: A is at the corner, and B is on the boundary

+D+ >=8



Proof that OC is not harmful

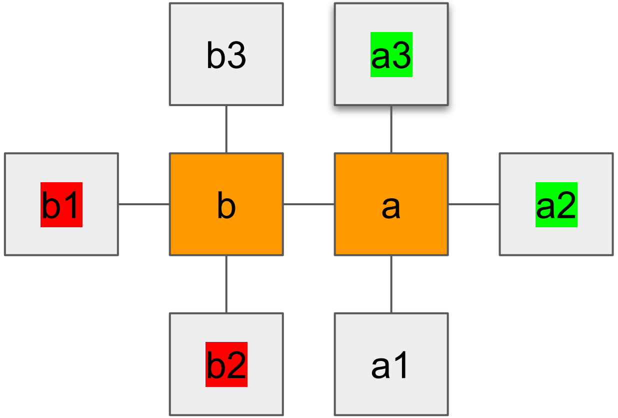
1. Directly prove that distance k 2-swap on optimal solution under OC will not decrease the HPWL.
2. If the 2-swap is not against OC, HPWL will not decrease since it is opt solution under OC.
3. If the 2-swap is against OC, there will be 3 basic relative position of the two swapped macro.

Basic distance 1 2-swap

Hpwl change = 0 for orange part,

B3,a3 will show up in pair

B2, a1 will show up in pair



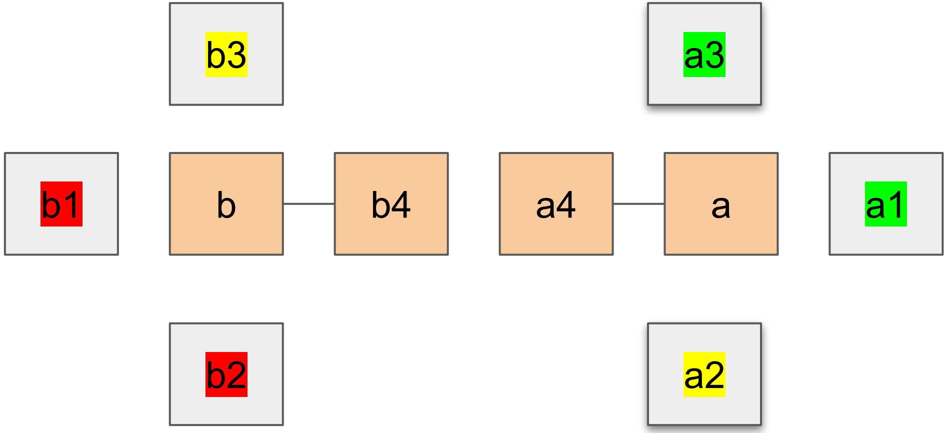
On the same row or column

Y(a)>Y(a4)>=Y(b4)>Y(b)

Focus on these 4 macros,

Swap a and b

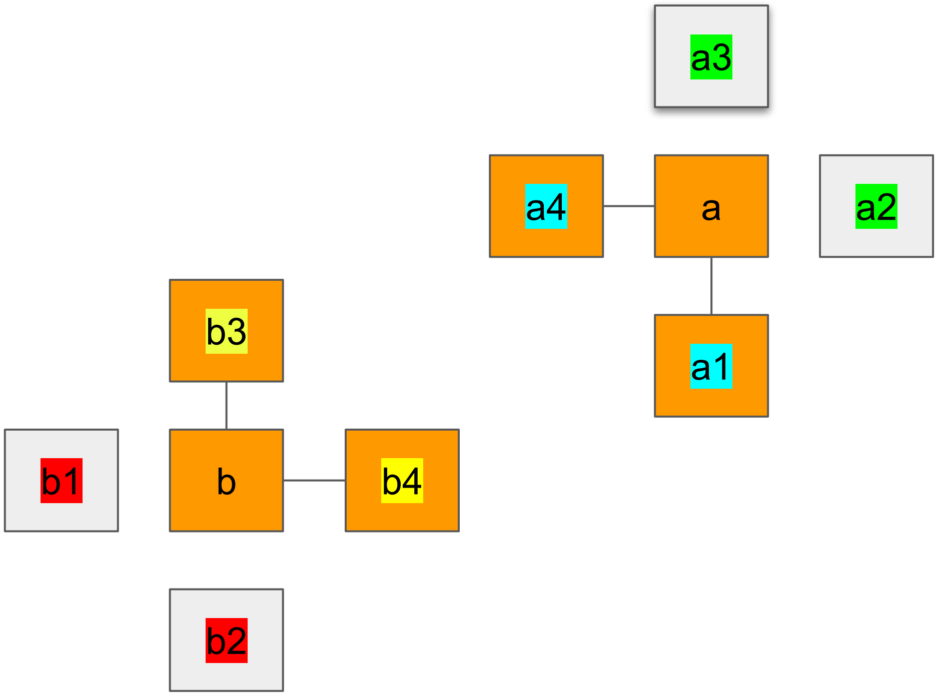
1. before=Y(b4)-Y(b) +Y(a)-Y(a4)
2. After = Y(a)-Y(b4)+Y(b4)-Y(b)
3. After - before = 2(Y(a4)-Y(b4)) >=0
4. A3 b3 will show up in pair
5. A2 b2 will show up in pair



On different rows/columns

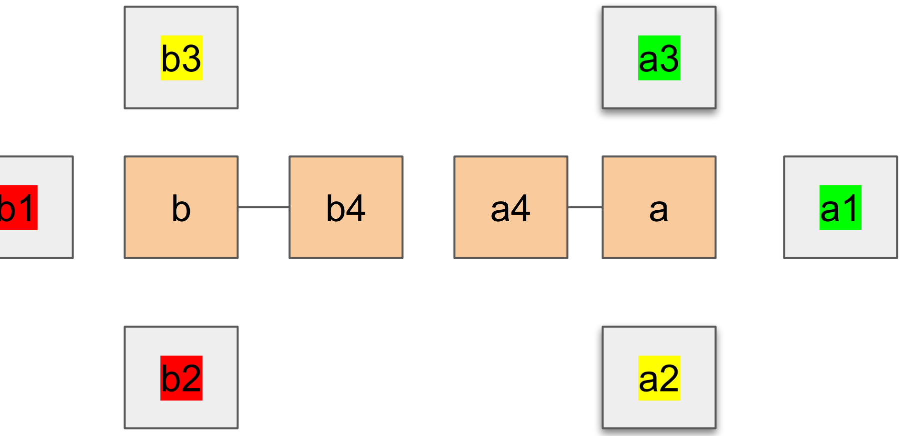
Focus on macro：a, b, a4, a1, b3, b4

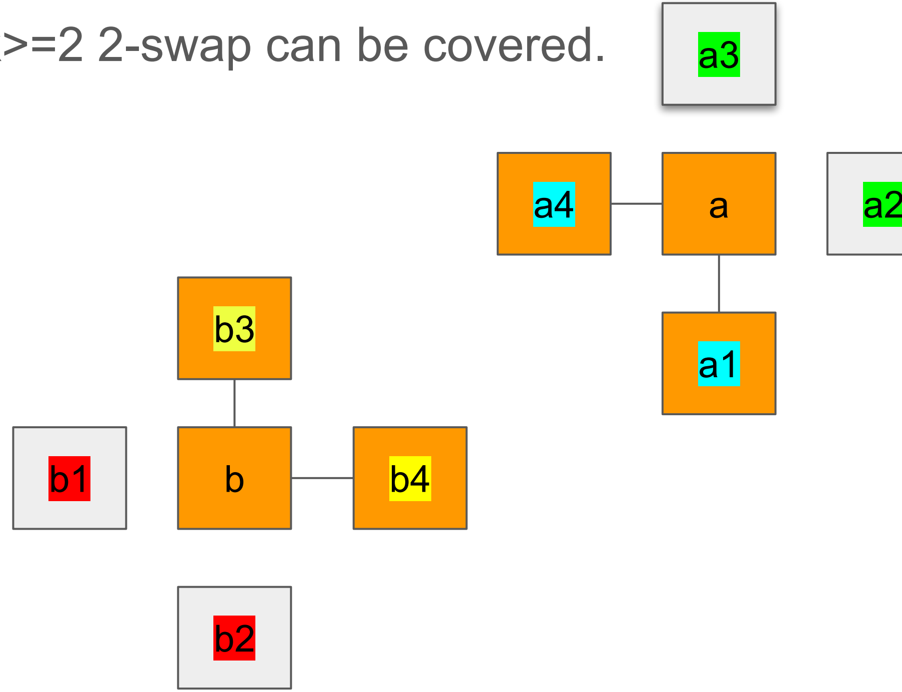
a. HPWL Before swap =2Y(a)-2Y(b)+Y(b3)+Y(b4)-Y(a4)-Y(a1)
b. HPWL After swap = 2Y(a) -2Y(b) -Y(b3)-Y(b4) +Y(a4)-Y(a1)
c. After-before = 2(Y(a4)+Y(a1)-Y(b3)-Y(b4)) >=0,
d. equal when a4 = b3, b4 = a1



Extend to other situation

From these 2 basic structures, all distance k>=2 2-swap can be covered.





Conclusion

Optimal solution under OC is local optimal solution without OC

According to the 3 basic structures, if min(m,h)>4, there is no HPWL non-increase 2-swap chain can bridge an optimal solution under OC to an optimal solution against OC.

Further : prove RSAD is optimal

Still hard: first how to prove RSAD is optimal under OC?

But the DP/mcmf based method can be easily proved as optimal solution under OC

Breakdown OC ：Monotonic within each row/column

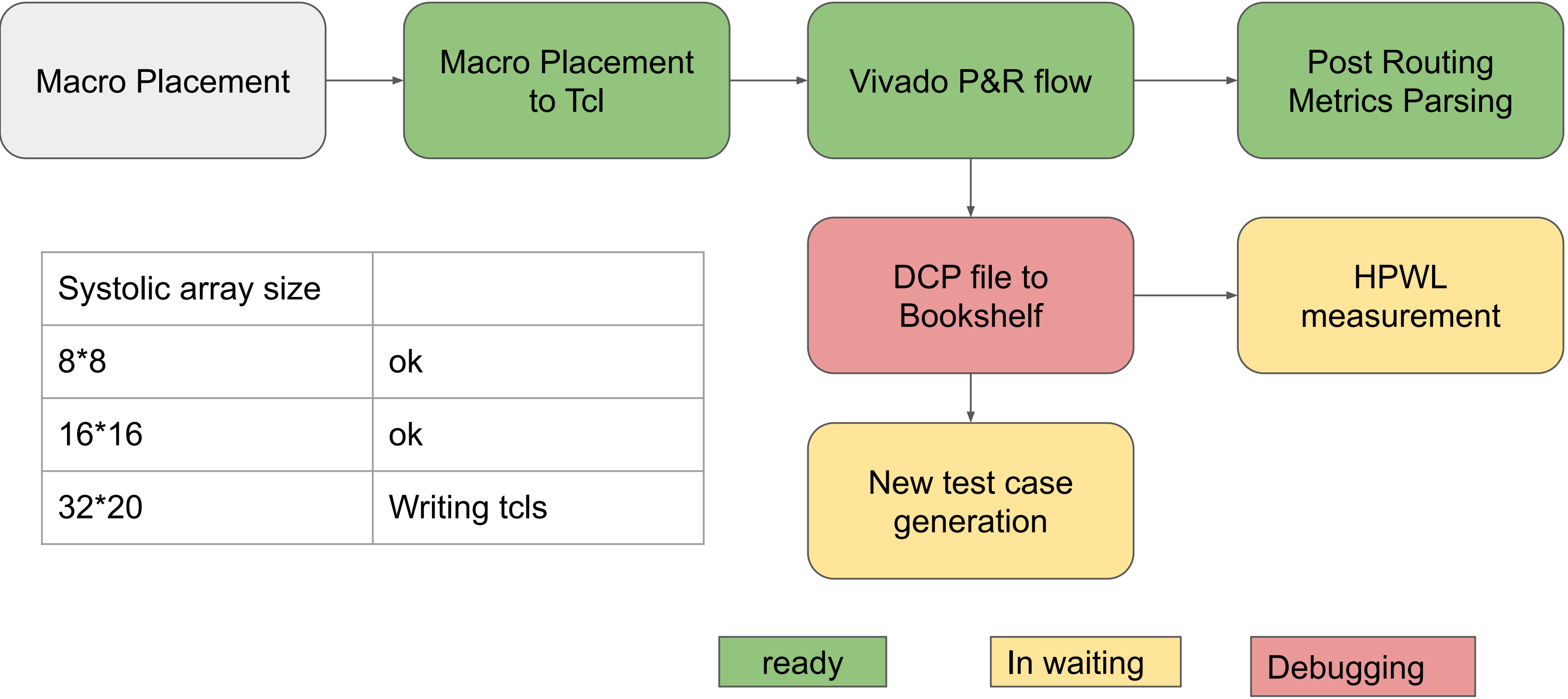
Local OC: Monotonic within each row/column.

Global OC: original OC

We can prove the optimal solution must be monotonic within each row/column.

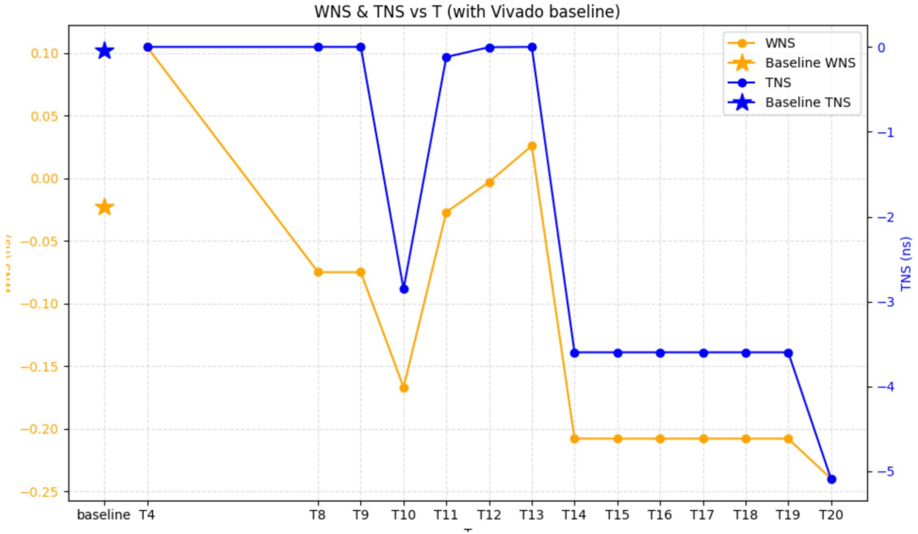
And prove the optimal solution must obey local OC, and also has equivalent global OC placement in HPWL.

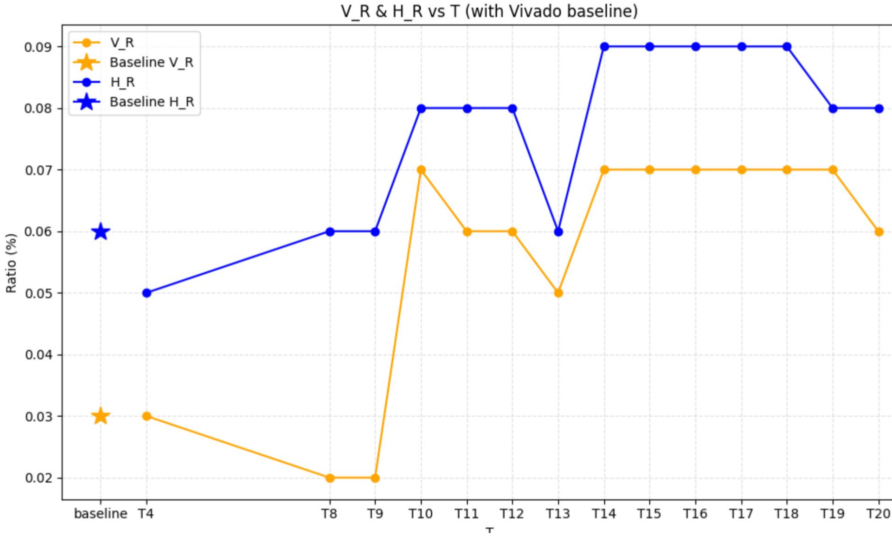
hailiang’s evaluation flow

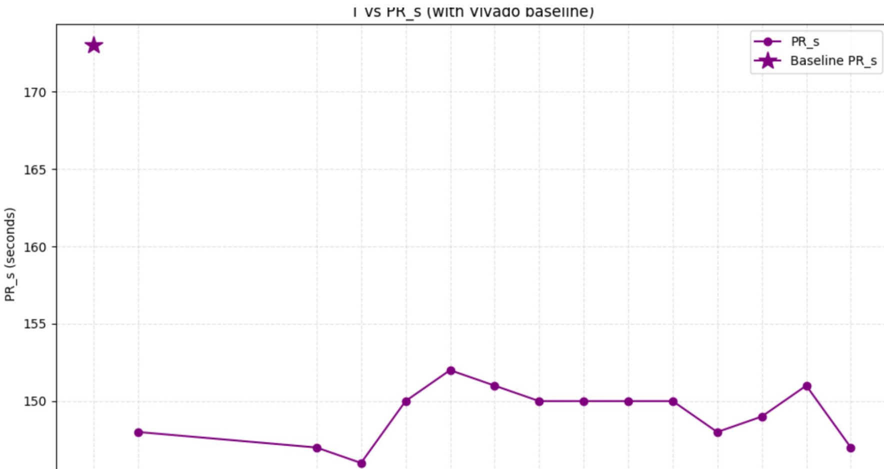


Systolic array size	
8*8	ok
16*16	ok
32*20	Writing tcls

Result for 8*8 systolic array



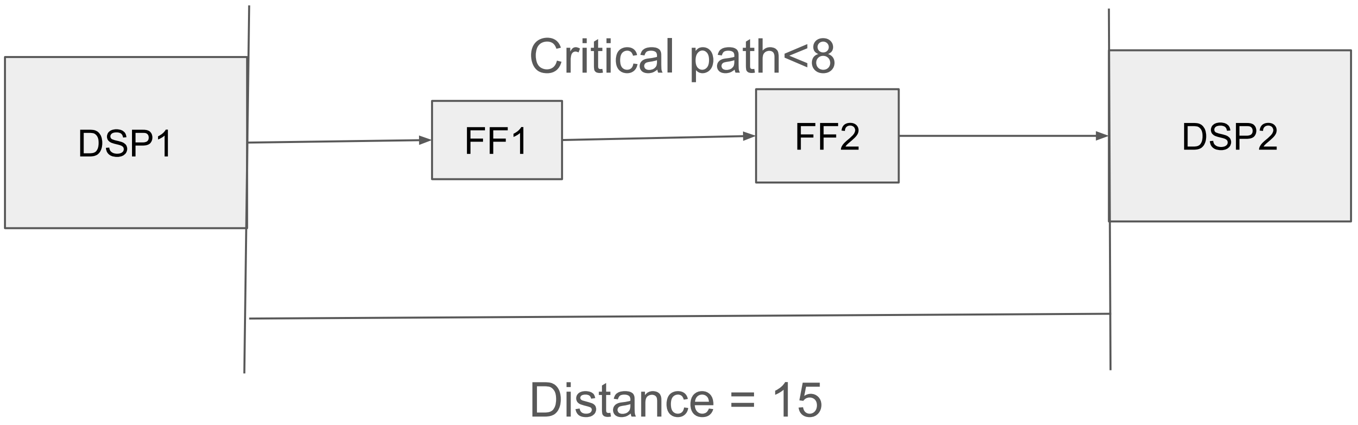




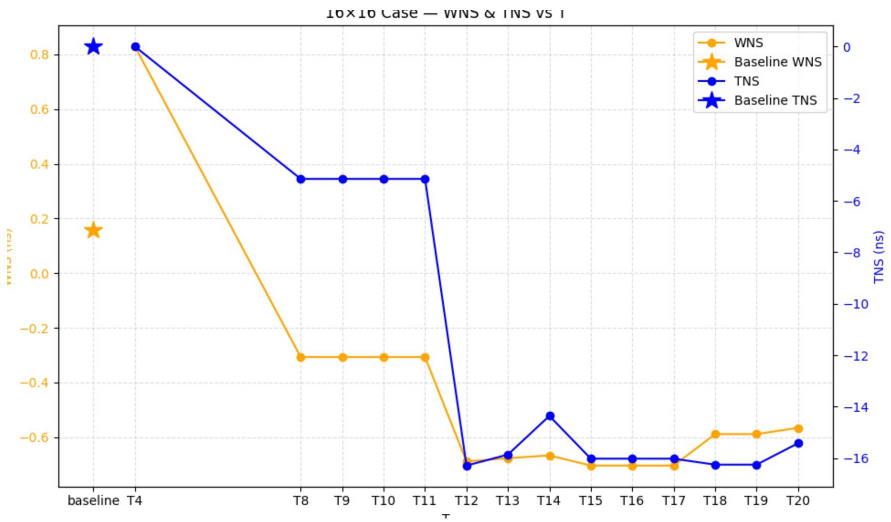
Why place DSP to 2 cols is better in WNS/TNS?
T = max WL in each column

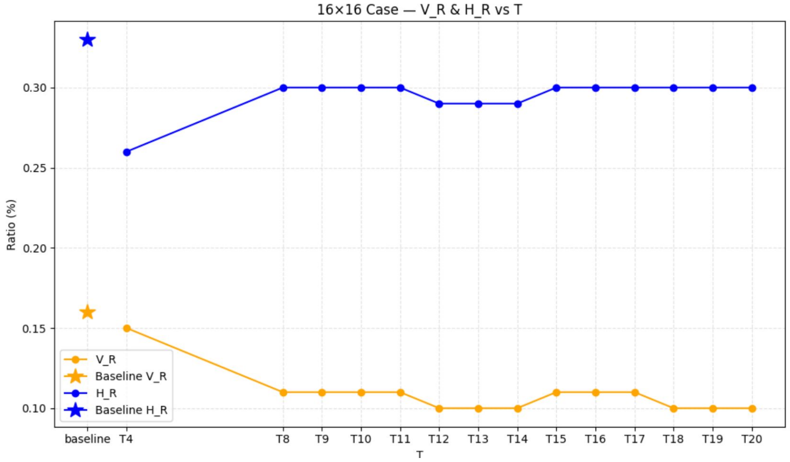
Horizontal delay is much shorter than our assumption

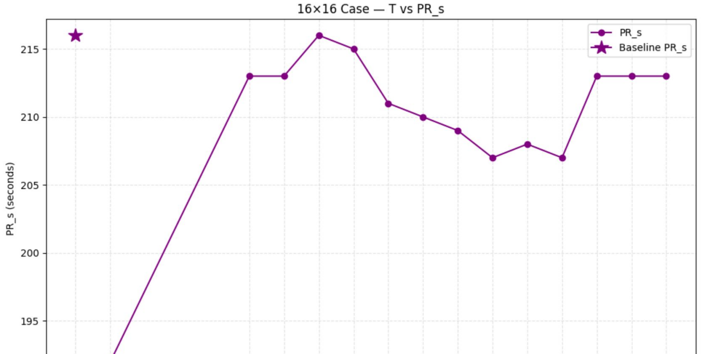
1. We assume T_horizon = 15
2. But the result shows that T_horizon < 8
3. Possible reason: DSPs are not directly connected, and there are some FFs in the path.



Results for 16*16







Placement on 4 columns is better, (T=4)

g=1 no room to optimize max WL

When g=1, max WL = min(m,h) = lower bound

When g!=1, then we have a chance to win RSAD in timing

If we assume T_horison = 4, then

The smallest test case where g!=1: 6*24

And 24*24 is the smallest square test case we can win RSAD

Then we can test on: 16*16(as good as RSAD)

24*24 =576 , 32*32=1024, 40*40=1600, where we can win

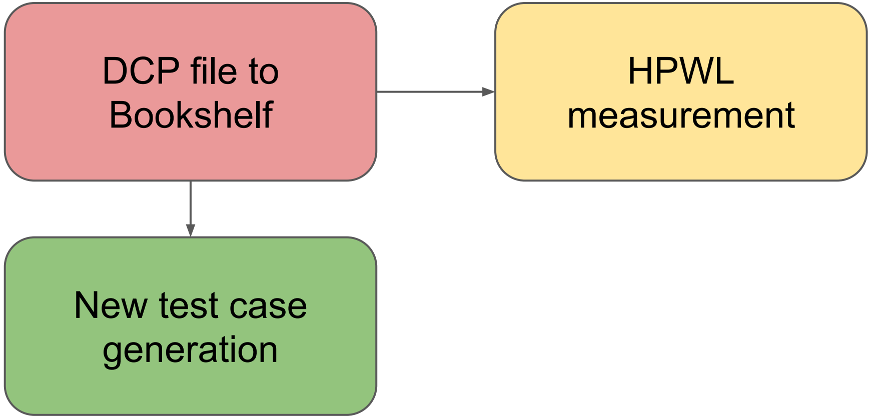
FPGA1 has 678 DSP sites.

FPGA2 has 1800 DSP sites.

New testcases

Thanks to Giacomo's hard work, the large test cases are ready to use.

Systolic array size	
16*16	ok
24*24	ok
32*32	ok
40*40	ok



ready

In waiting

Debugging

Runtime and converge issue on large circuit

1. For 32*32/40*40, each iteration runtime is around 1 min, and can not converge.

Quick try - state compression

1. If a state has only one downstream state, then it can be merged.
2. If a state has a downstream state dominate other state, then it can be merged.
3. If a state has several downstream states and choosing any of them can lead to the optimal solution, then it can be merged.

=== Exact-Compressed DP on 12x12 (equal weights) ===

Variant	BestCost	Time(s)	Expansions	PeakStates
baseline	1568	25.500	16224936	61108
forced-run	1568	52.856	16224793	61104
forced+rot180	1568	57.799	8124679	30586

Solution: what kind of states are compressed

1. RSAD: the CTR region

Logical layout (labels=y)
placement8x8_Col_1_T_20.txt

0	2	4	24	32	40	43	45
1	3	5	25	33	41	44	46
6	7	8	26	34	42	47	48
9	10	11	27	35	49	50	51
12	13	14	28	36	52	53	54
15	16	17	29	37	55	58	59
18	20	22	30	38	56	60	62
19	21	23	31	39	57	61	63

2. With Max WL <=T const

Logical layout (labels=y)
placement8x8_Col_1_T_11.txt

0	1	2	3	4	10	12	13
5	6	7	8	9	11	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	50	52	54	55	56	57	58
49	51	53	59	60	61	62	63

Logical layout (labels=y)
placement8x8_Col_1_T_14.txt

0	2	16	24	32	40	48	50
1	3	17	25	33	41	49	51
4	5	18	26	34	42	52	53
6	7	19	27	35	43	54	55
8	9	20	28	36	44	56	57
10	11	21	29	37	45	58	59
12	13	22	30	38	46	60	62
14	15	23	31	39	47	61	63

More examples: on 16*8 systolic array

0	1	4	6	8	10	12	13
2	3	5	7	9	11	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87
88	89	90	91	92	93	94	95
96	97	98	99	100	101	102	103

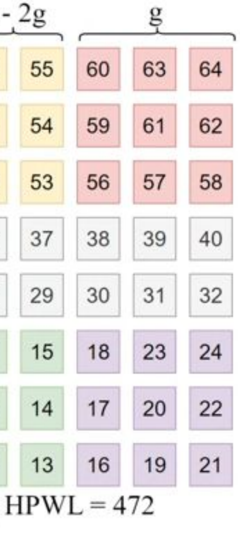
T=14

0	2	4	9	12	15	17	18
1	3	5	10	13	16	19	20
6	7	8	11	14	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87
88	89	90	91	92	93	94	95
96	97	98	99	100	101	102	103
104	105	110	113	116	119	122	123
106	108	111	114	117	120	124	126
107	109	112	115	118	121	125	127

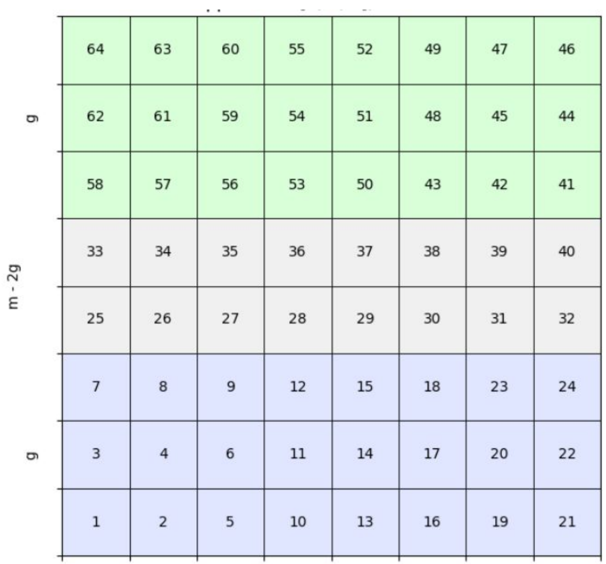
T=20

Observation: independent CTR region

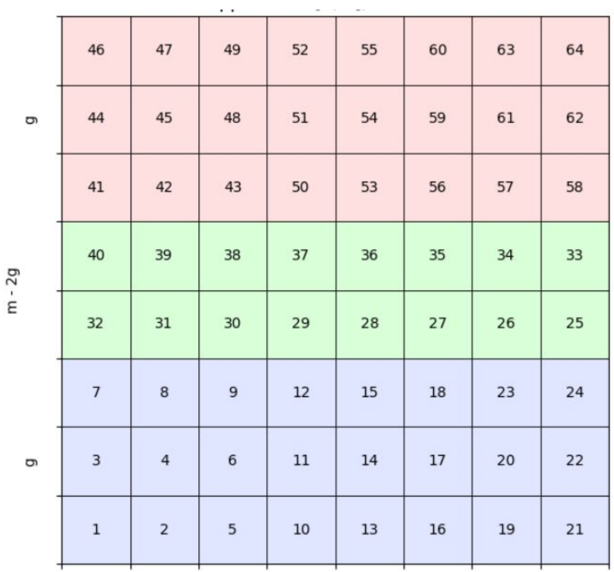
The HPWL is as same as RSAD result.



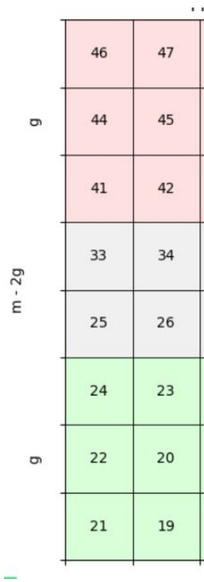
Flip top 3 rows.



Flip mid 2 rows.

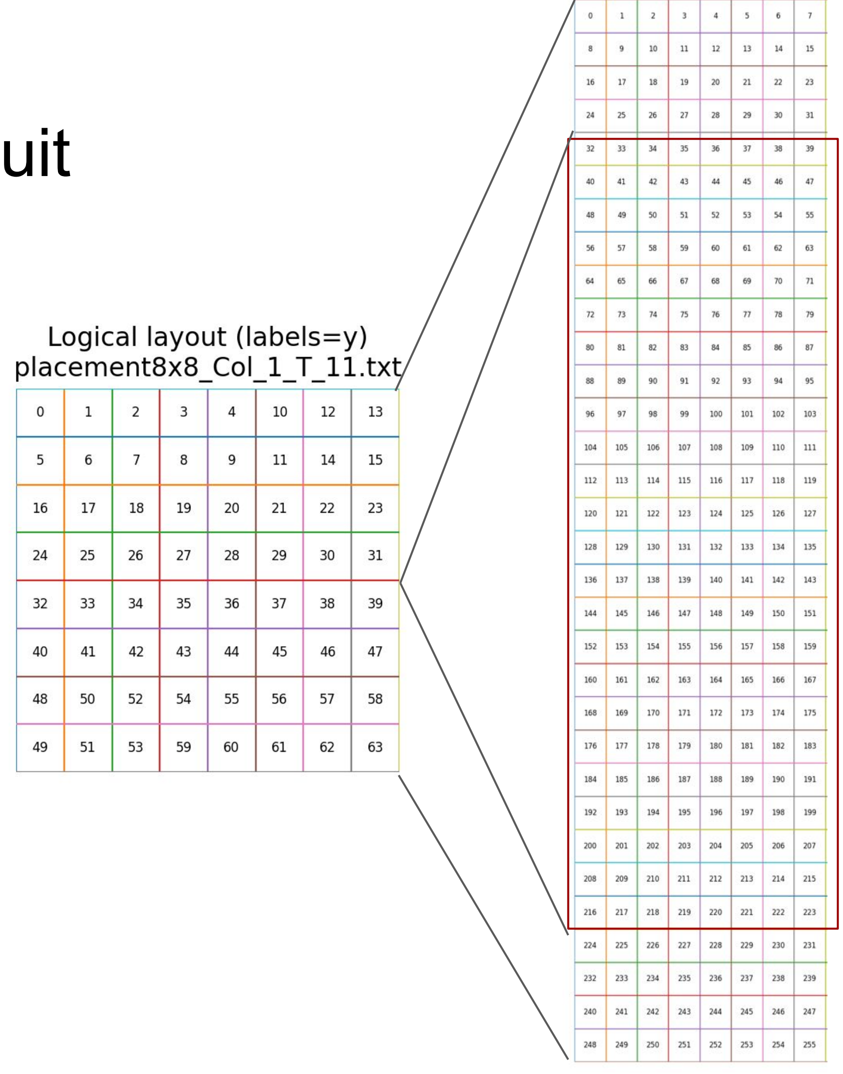


Flip b



Partition the large circuit

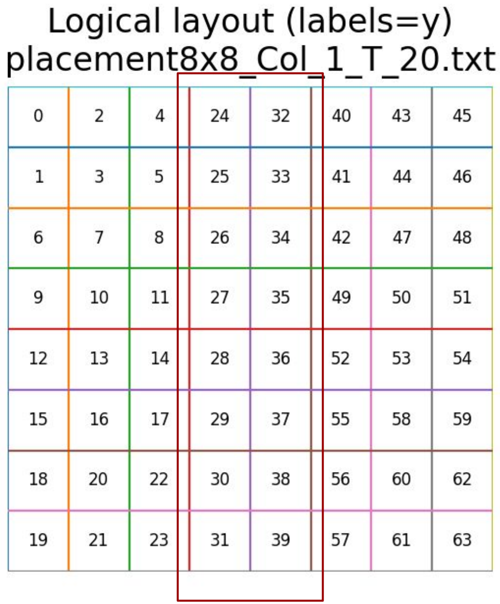
1. Given a m*h macro array
2. Partition it into 3 parts
3. Top(h/2 rows)
4. Bot(h/2 rows)
5. CTR(the rest)
6. Solve the h*h problem
7. Restore the m*h solution



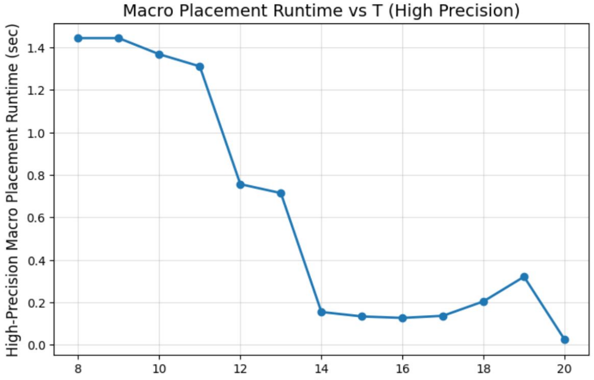
Compress the CTR region

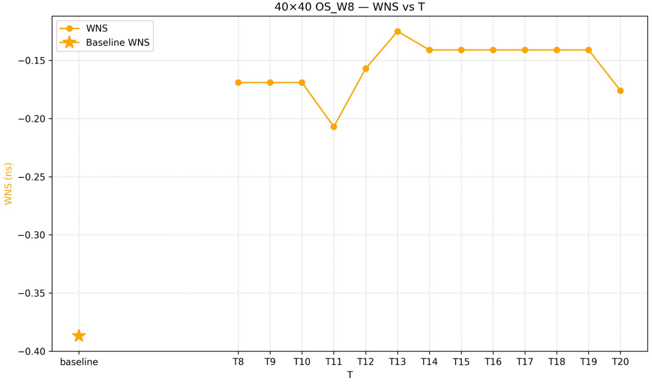
Rotate the solution

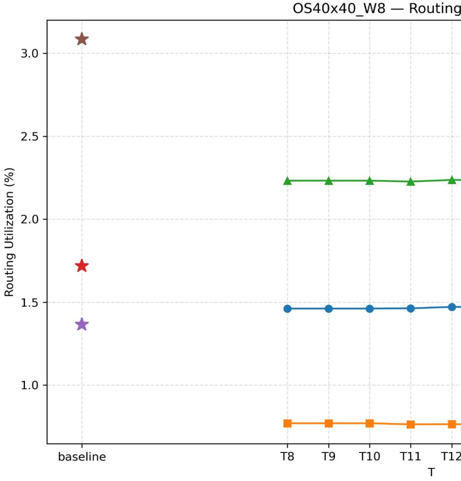
1. If the largest element in top 4 rows is larger then m*h/2
2. Then rotate the solution

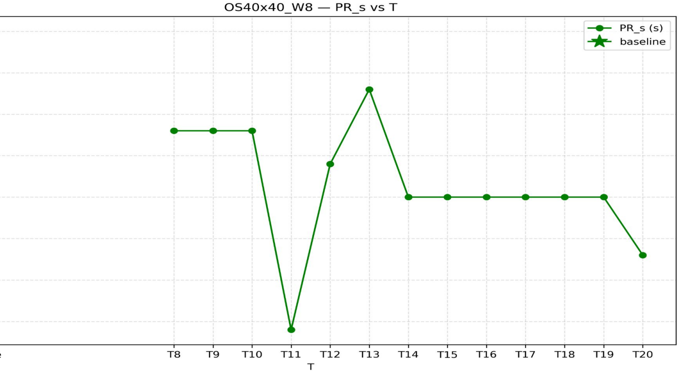


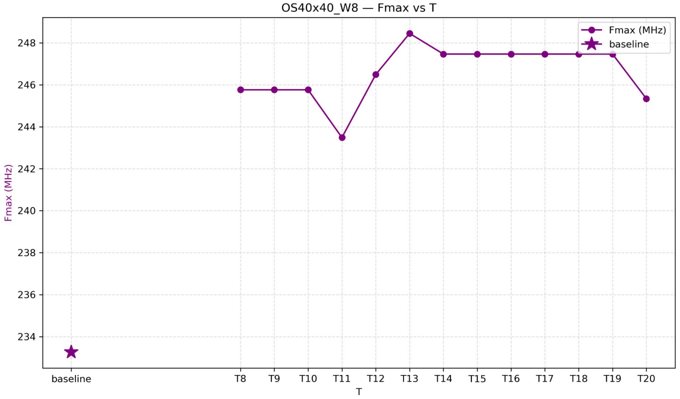
Result on 40*40 Fmax 6% increase Route util -27.3%



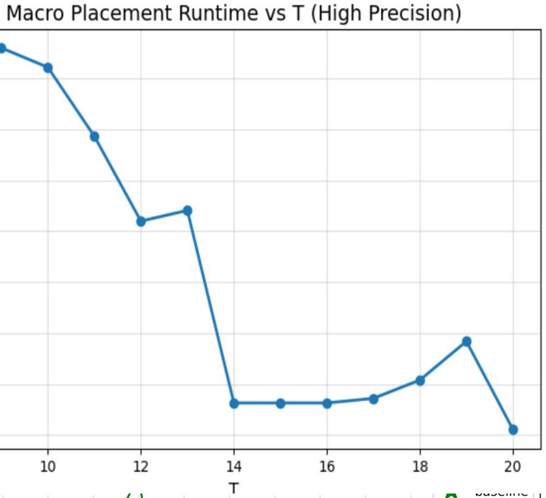


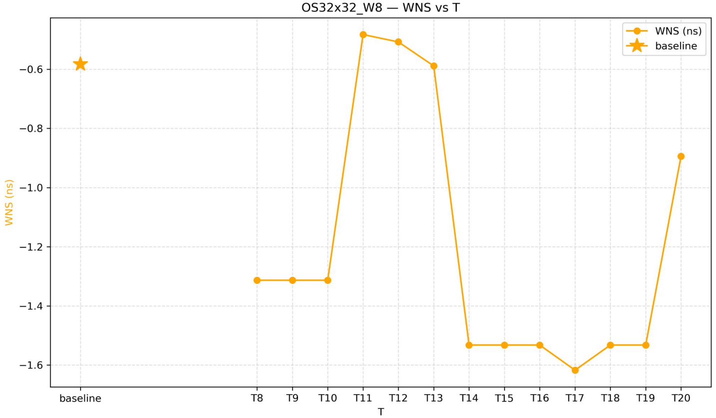


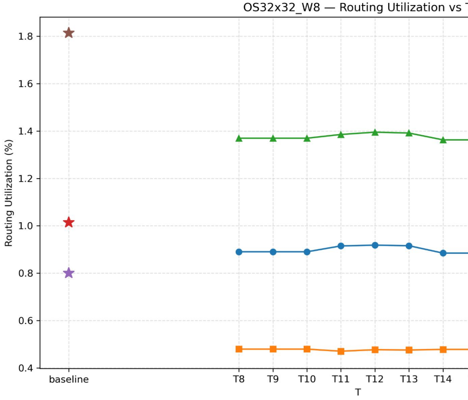


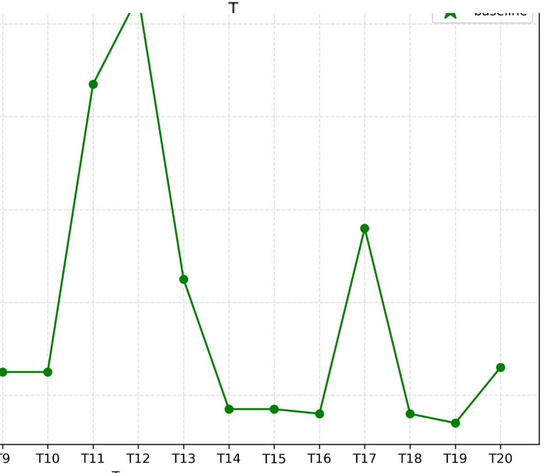


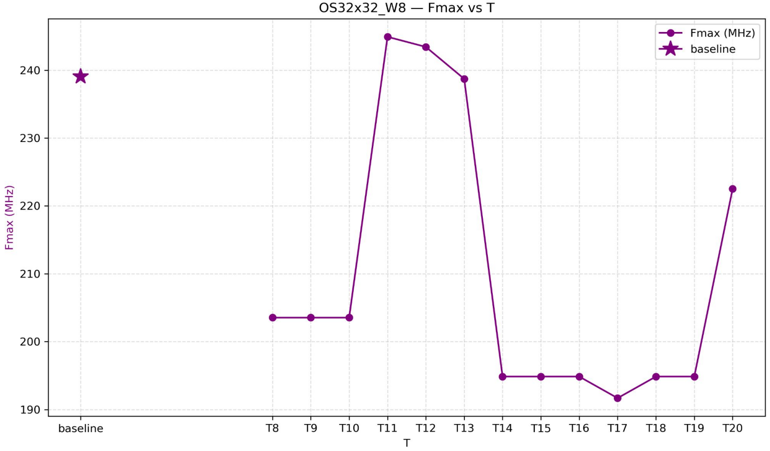
Result on 32*32 Fmax 3% increase Route_util -23.2%



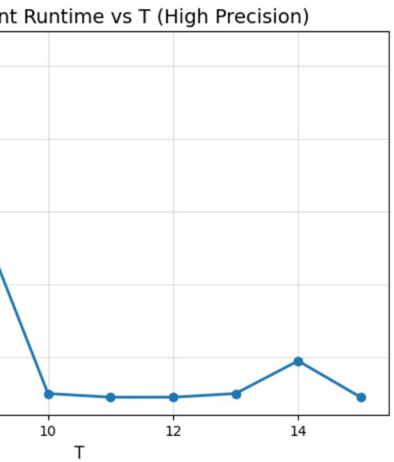


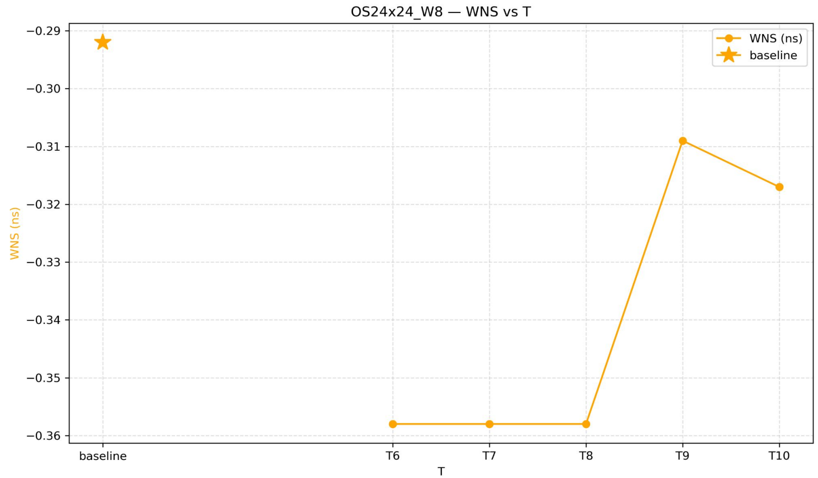


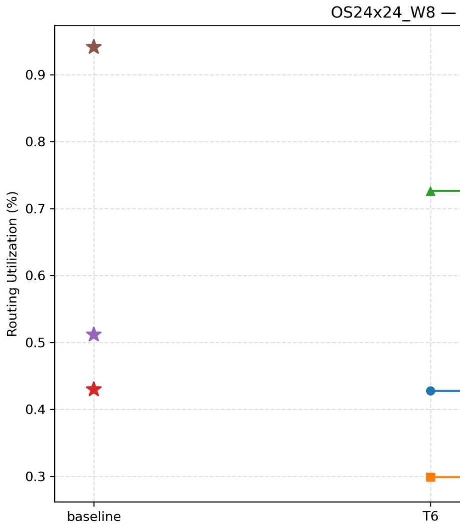


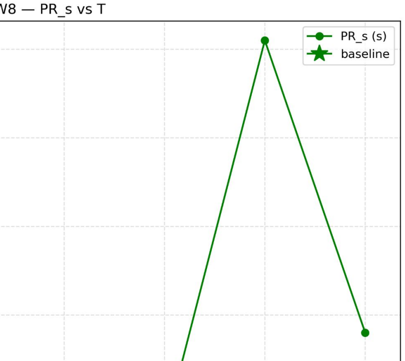


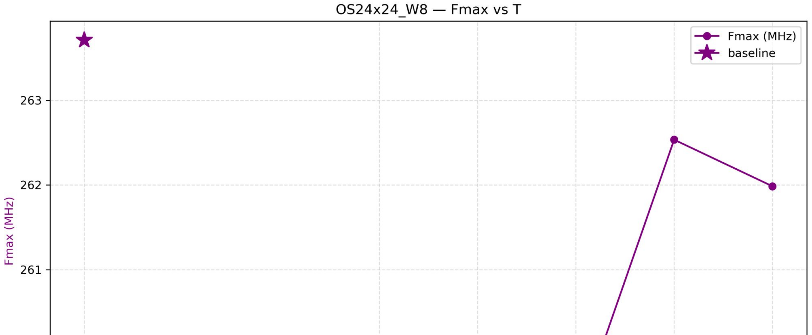
Result on 24*24 Fmax -0.45% Route_util -22.5%











To do

1. Prove：

   For any given input dimensions m,h, and any timing bound T, if the optimal T-feasible placement for (m,h) contains a Central Region (CTR), then the optimal solution for the enlarged instance (m+1,h) under the same timing bound T is obtained by extending the CTR upward by exactly one additional row.

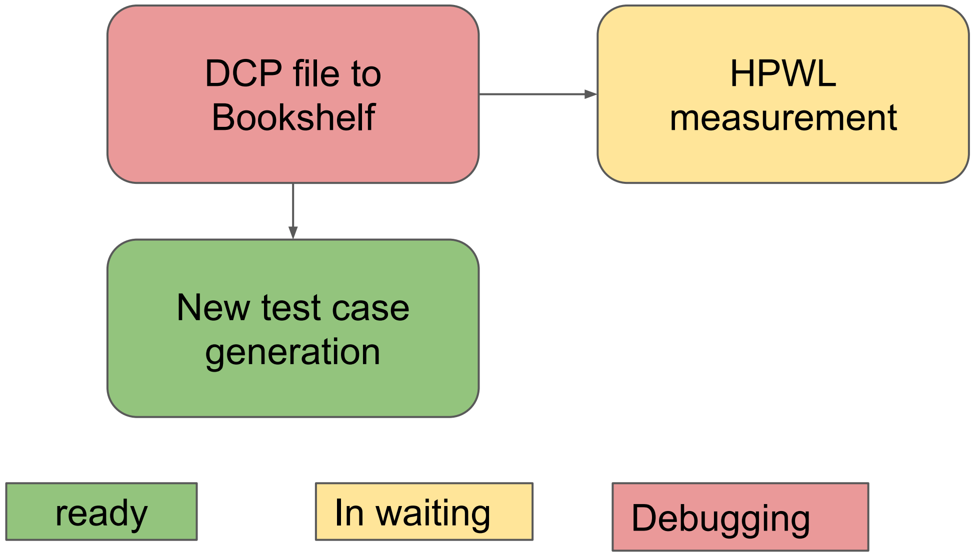
To do

1. Implement DSPplacer(DAC'25)
2. Run Dreamplace-fpga, AMF-place 2.0

Large testcases

Thanks to Giacomo's hard work, more large test cases are ready to use.

Systolic array size	
24*24	ok
32*32	ok
40*40	ok
36*32	ok
40*36	ok
40*32	ok
44*40	ok



Experiment result

case	method	T_value	WNS(ns)	TNS(ns)	Fmax(MHz)	V_R(%)	H_R(%)	Tot_R(%)	Total CPU
OS24x24_W8	RSAD	10	-0.32	-297.27	261.99	0.44	0.30	0.73	338
	Vivado		-0.29	-273.47	263.71	0.43	0.51	0.94	355
	ours	9	-0.31	-351.68	262.54	0.44	0.29	0.73	371
OS32x32_W8	RSAD	20	-0.89	-2289.01	222.52	0.88	0.47	1.34	626
	Vivado		-0.58	-1298.78	239.06	1.01	0.80	1.81	696
	ours	11	-0.48	-1455.69	244.92	0.92	0.47	1.39	687
OS36x32_W8	RSAD	20	-0.46	-1002.22	246.61	1.36	0.49	1.85	609
	Vivado		-1.34	-1537.04	202.51	1.13	0.94	2.07	561
	ours	10	-0.44	-988.01	247.71	1.38	0.50	1.88	585
OS40x32_W8	RSAD	20	-1.44	-2285.96	198.41	1.08	0.61	1.69	571
	Vivado		-0.74	-1564.21	230.41	1.31	1.21	2.52	811
	ours	12	-0.49	-1239.85	244.56	1.15	0.62	1.76	599
OS40x36_W8	RSAD	20	-0.45	-1115.90	246.79	1.32	0.69	2	642
	Vivado		-0.59	-1442.15	238.55	1.47	1.35	2.82	690
	ours	11	-0.39	-1055.06	250.50	1.32	0.70	2.02	663
OS40x40_W8	RSAD		-0.18	-22.46	245.34	1.47	0.76	2.23	693
	Vivado		-0.39	-39.24	233.26	1.72	1.37	3.08	720
	ours	13	-0.13	-12.35	248.45	1.47	0.76	2.24	713
OS44x40_W8	RSAD	20	-0.53	-1708.49	242.25	2.11	0.76	2.86	1098
	Vivado		-1.03	-3026.82	215.84	2.19	1.75	3.94	1053
	ours	13	-0.48	-1567.06	244.86	2.14	0.76	2.9	1089
AVG	RSAD		-0.61	-1245.90	237.70	1.24	0.58	1.81	653.86
	Vivado		-0.71	-1311.67	231.91	1.32	1.13	2.45	698.00
	ours		-0.39	-952.81	249.08	1.26	0.59	1.85	672.43
Norm	RSAD		156.75%	130.76%	95.43%	98.19%	99.51%	98.30%	97.24%
	Vivado		182.60%	137.66%	93.11%	104.99%	193.41%	132.97%	103.80%
	ours		100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%

How to determine T?

Option 1: user-defined

Option 2: min HPWL+w* enumerate all macro placement solution and then determine T, runtim is around 10s

If run 24*24 on fpga1(768 dsp sites)

case	method	T_value	clk(ns)	WNS(ns)	TNS(ns)	Fmax(MHz)	|WNS|/T(%)	V_R(%)	H_R(%)	Tot_R(%)	Synth_s	Total CPU
OS24x24_W8	RSAD	10	3.6	-1.059	-2890.913	214.64	29.42	0.86	0.66	1.52	81	381
	Vivado		3.6	-1.056	-3017.519	214.78	29.33	1.12	0.75	1.87	82	386
	ours	6	3.6	-1.04	-2935.131	215.52	28.89	0.84	0.66	1.5	85	387

Experiment setup

	Macro Placer	Std cell Placer	Evaluator
Ours	Our algo	Vivado	Bookshelf->HPWL,vivado-> WNS/TNS/Fmax
DreamplaceFpga-Mp	DreamplaceFpga-Mp	vivado	Bookshelf->HPWL,vivado-> WNS/TNS/Fmax
DSPlacer	DSPlacer	AMFPlace 2.0	Bookshelf->HPWL,vivado-> WNS/TNS/Fmax
AMFPlace 2.0	AMFPlace 2.0	AMFPlace 2.0	Bookshelf->HPWL,vivado-> WNS/TNS/Fmax

DSPlacer implementation

WIth HKU deepcode, auto implemented.

Case	Runtime(s)	HPWL	MaxNet
24x24	0.689	6745	42
32x32	1.899	14647	52
40x40	3.21	17412	19
40x44	3.706	26716	60
Avg	2.376	16380	43.25

Macro placement comparison

Case	DSPlacer HPWL	DSPlacer MaxNet	RSAD HPWL	RSAD MaxNet	Ours HPWL	Ours MaxNet
24x24	6745	42	4832	10	4840	9
32x32	14647	52	10144	20	10208	11
40x40	17412	19	16120	20	16150	13
40x44	26716	60	17780	20	17810	13
Avg	16380	43.25	12219	17.5	12252	11.5
Norm (%)	135.2	376.1	99.7	152.2	100	100

Errors detected in DCP->XDC->Bookshelf->HPWL flow

1. Device not match, fixed
2. Netlist not match, fixed in one case
3. Component name, like DSP name not match, fixed in one case
4. Coordinate parse error, fixed

case	hpwl	sHpwl	vs_vivado_default
32x32_col4_T8	130933	143484	-42.60%
32x32_col4_T11	158029	176993	-30.70%
32x32_col4_T12	157612	176237	-30.90%
32x32_col4_T14	130319	144117	-42.80%
32x32_col4_T15	130319	144117	-42.80%
32x32_col4_T16	130319	144117	-42.80%
32x32_col4_T17	103408	112002	-54.60%
32x32_col4_T18	130319	144117	-42.80%
32x32_col4_T19	130319	144117	-42.80%
RSAD	101429	110505	-55.50%
Vivado_default	228016	238663	baseline

1. Need further check
2. If placement is legal

Case	Method	HPWL	Routed_WL	WNS	TNS	Fmax	Tot_R	CPU
OS24x24_W8	Vivado	245232	385709	-0.29	-273.47	263.71	0.94	355
	RSAD	177188	370829	-0.32	-297.27	261.99	0.73	338
	Ours	177441	370419	-0.31	-351.68	262.54	0.73	371
OS32x32_W8	Vivado	494792	713100	-0.58	-1298.78	239.06	1.81	696
	RSAD	356790	662786	-0.89	-2289.01	222.52	1.34	626
	Ours	376203	667613	-0.48	-1455.69	244.92	1.39	687
OS36x32_W8	Vivado	555410	809912	-1.34	-1537.04	202.51	2.07	561
	RSAD	568842	801218	-0.46	-1002.22	246.61	1.85	609
	Ours	578173	808002	-0.44	-988.01	247.71	1.88	585
OS40x32_W8	Vivado	690842	914692	-0.74	-1564.21	230.41	2.52	811
	RSAD	448668	819568	-1.44	-2285.96	198.41	1.69	571
	Ours	475704	837851	-0.49	-1239.85	244.56	1.76	599
OS40x36_W8	Vivado	774885	1029189	-0.59	-1442.15	238.55	2.82	690
	RSAD	540131	945011	-0.45	-1115.9	246.79	2	642
	Ours	539512	946824	-0.39	-1055.06	250.5	2.02	663
OS40x40_W8	Vivado	854961	1138779	-0.39	-39.24	233.26	3.08	720
	RSAD	597408	1050282	-0.18	-22.46	245.34	2.23	693
	Ours	593826	1046746	-0.13	-12.35	248.45	2.24	713
OS44x40_W8	Vivado	1167074	1310019	-1.03	-3026.82	215.84	3.94	1053
	RSAD	875596	1230002	-0.53	-1708.49	242.25	2.86	1098
	Ours	892266	1236053	-0.48	-1567.06	244.86	2.9	1089
AVG	Vivado	683314	900200	-0.71	-1311.67	231.91	2.45429	698
	RSAD	509232	839957	-0.61	-1245.90	237.70	1.81429	653.857
	Ours	519018	844787	-0.39	-952.81	249.08	1.84571	672.429
Norm(Avg)	Vivado	131.66%	106.56%	182.35%	137.66%	93.11%	132.97%	103.80%
	RSAD	98.11%	99.43%	156.99%	130.76%	95.43%	98.30%	97.24%
	Ours	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%